

Скоринговая модель для микрофинансирования

Пет-проект по машинному обучению
Датасет: Home Credit Default Risk (Kaggle)

1. Постановка задачи

Бизнес-задача. Микрофинансовая организация выдаёт займы, в том числе клиентам с тонкой или отсутствующей кредитной историей. Требуется оценить вероятность того, что заёмщик допустит просрочку по платежам, на основе данных анкеты, заполняемой при подаче заявки.

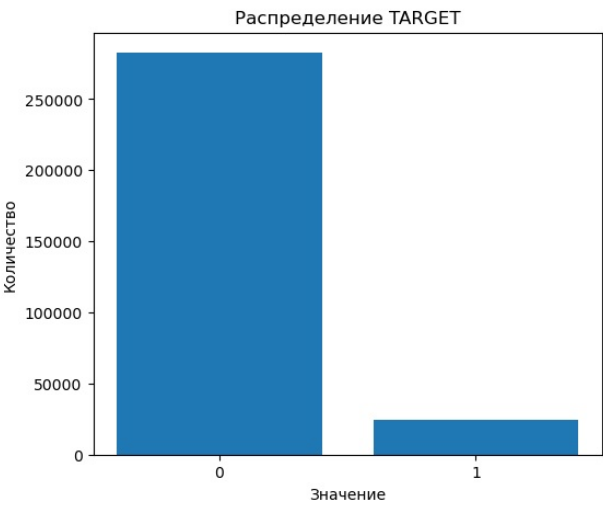
Формальная постановка. Бинарная классификация. Объект — заявка на кредит, описанная 120 признаками. Целевая переменная `TARGET` : 1 — клиент допустил просрочку, 0 — платил без задержек.

Данные. Таблица `application_train.csv` из соревнования Home Credit Default Risk: 307 511 строк, 122 колонки. Колонка `SK_ID_CURR` — технический идентификатор заявки, удаляется сразу.

Состав признаков: 65 вещественных, 40 целочисленных, 16 строковых (категориальных). Содержательно они делятся на несколько блоков: демография заёмщика, параметры запрашиваемого кредита, характеристики места работы и жилья, флаги предоставленных документов, количество запросов в кредитное бюро и три внешних кредитных рейтинга `EXT_SOURCE_1/2/3`.

Метрика — ROC-AUC, задана условиями соревнования.

Баланс классов



Распределение целевой переменной

Дефолты составляют **8.07%** выборки (24 825 против 282 686) — примерно один проблемный заёмщик на одиннадцать добросовестных.

2. Разведочный анализ и предобработка

2.1 Пропуски

Пропуски присутствуют в **67 колонках из 120** и сосредоточены в двух смысловых блоках:

Блок признаков	Доля пропусков	Интерпретация
<code>COMMONAREA_*</code>	69.9%	Данные о доме заёмщика собирались не всегда
<code>NONLIVINGAPARTMENTS_*</code>	69.4%	То же
<code>FONDKAPREMONT_MODE</code>	68.4%	То же
<code>LIVINGAPARTMENTS_*</code>	68.4%	То же
<code>FLOORSMIN_*</code>	67.8%	То же
<code>YEARS_BUILD_*</code>	66.5%	То же
<code>EXT_SOURCE_1</code>	~56%	Внешний рейтинг доступен не для всех клиентов
<code>OCCUPATION_TYPE</code>	~31%	Пропуск фактически означает «не работает»
<code>EXT_SOURCE_3</code>	~20%	Внешний рейтинг недоступен

Заполнение пропусков вынесено **внутри пайплайна** (`SimpleImputer` как шаг `ColumnTransformer`), а не выполняется заранее по всему датафрейму. Это принципиально: при заполнении «до сплита» среднее и медиана считались бы по всей выборке, включая объекты валидационного фолда, что даёт утечку данных и завышенную оценку качества.

Стратегии заполнения выбраны по типу признака:

- **бинарные** (ровно два уникальных значения, 32 признака) — медиана, чтобы результат остался в {0, 1};
- **остальные числовые** (77 признаков) — среднее;
- **категориальные** (16 признаков) — мода, затем One-Hot с `handle_unknown='ignore'` на случай категории, встретившейся только в валидационном фолде.

Проверка на константные колонки (`nunique <= 1`) не выявила ни одной — все признаки несут вариативность.

2.2 Выбросы

По боксплотам всех числовых признаков выявлены распределения с тяжёлыми хвостами. Три признака обрезаны по 1-му и 99-му перцентилю: `AMT_INCOME_TOTAL`, `AMT_CREDIT`, `CNT_FAM_MEMBERS`. Наиболее выраженный случай — `AMT_INCOME_TOTAL`, где встречаются значения на два порядка выше медианы.

Клиппинг нужен прежде всего линейной модели и нейросети: экстремальные значения искажают масштабирование и дестабилизируют градиенты. Деревья решений к выбросам почти нечувствительны, так как используют только порядок значений, а не их величину.

2.3 Категориальные признаки

Из 16 строковых колонок две имеют высокую кардинальность: `ORGANIZATION_TYPE` (58 категорий) и `OCCUPATION_TYPE` (18). Остальные содержат менее 10 уникальных значений. Все категориальные признаки закодированы через One-Hot.

2.4 Конструирование признаков

Ключевое наблюдение: абсолютные денежные суммы сами по себе малоинформативны. Кредит в 1 млн означает принципиально разный уровень риска для заёмщика с доходом 200 тыс. и с доходом 2 млн. Поэтому сконструированы пять **относительных** признаков — именно такими показателями оперируют кредитные аналитики:

Признак	Формула	Обоснование
<code>CREDIT_INCOME_RATE</code>	<code>AMT_CREDIT / AMT_INCOME_TOTAL</code>	Во сколько раз кредит превышает годовой доход
<code>ANNUITY_INCOME_RATE</code>	<code>AMT_ANNUITY / AMT_INCOME_TOTAL</code>	Долговая нагрузка — ключевой показатель скоринга
<code>CREDIT_ANNUITY_RATIO</code>	<code>AMT_CREDIT / AMT_ANNUITY</code>	Фактический срок кредита в периодах
<code>CREDIT_GOODS_RATIO</code>	<code>AMT_CREDIT / AMT_GOODS_PRICE</code>	Переплата, косвенный признак размера первого взноса
<code>INCOME_PER_PERSON</code>	<code>AMT_INCOME_TOTAL / CNT_FAM_MEMBERS</code>	Доход на одного члена семьи

Технический нюанс: деление даёт `inf` там, где знаменатель равен нулю (в `AMT_ANNUITY` встречаются нулевые значения). Бесконечности заменены на `NaN` до передачи в пайплайн, иначе `SimpleImputer(strategy='mean')` вычислил бы среднее равным бесконечности и заполнил бы им весь столбец, полностью его обесценив.

После добавления новых признаков число числовых колонок выросло со 104 до **109**.

Проверка гипотезы. Все пять сконструированных признаков вошли в топ-20 по важности Random Forest (раздел 4.2), причём `CREDIT_ANNUITY_RATIO` оказался на 5-м месте — выше, чем исходные `AMT_ANNUITY` (12-е) и `AMT_CREDIT` (14-е), из которых он построен. `CREDIT_GOODS_RATIO` дополнительно попал в топ-5 по линейной корреляции с целевой переменной (0.064), опередив все признаки регионального рейтинга. Гипотеза об информативности отношений подтвердилась.

2.5 Корреляции с целевой переменной

Признак	Корреляция
<code>EXT_SOURCE_3</code>	−0.179
<code>EXT_SOURCE_2</code>	−0.160
<code>EXT_SOURCE_1</code>	−0.155
<code>DAYS_BIRTH</code>	+0.078
<code>CREDIT_GOODS_RATIO</code>	+0.064
<code>REGION_RATING_CLIENT_W_CITY</code>	+0.061
<code>REGION_RATING_CLIENT</code>	+0.059
<code>DAYS_LAST_PHONE_CHANGE</code>	+0.055

Максимальная по модулю корреляция составляет всего 0.18. Это принципиальная характеристика задачи: **сигнал слабый и распределён по многим признакам**. Ни один отдельный признак не является решающим, и качество модели определяется её способностью собрать множество слабых сигналов. Именно поэтому итоговые значения ROC-AUC находятся в районе 0.75–0.77, а не 0.95, как бывает на задачах с сильным сигналом.

Три лидера — `EXT_SOURCE_1/2/3` — это агрегированные рейтинги из внешних кредитных бюро, то есть, по сути, готовые предсказания чужих скоринговых моделей. Отрицательный знак корреляции ожидаем: чем выше внешний рейтинг, тем ниже риск дефолта.

3. Методология эксперимента

Схема валидации: стратифицированная 5-фолдовая кросс-валидация (`StratifiedKFold`, `shuffle=True`, фиксированный `random_state`). Стратификация обязательна: при 8% позитивного класса случайное разбиение может дать фолды с заметно различающейся долей дефолтов.

Защита от утечки данных. Вся предобработка (заполнение пропусков, One-Hot, масштабирование, отбор признаков) оформлена шагами `Pipeline`. При кросс-валидации `sklearn` обучает пайплайн заново на каждом `train`-фолде, поэтому все статистики считаются только по обучающей части.

Отбор признаков: `SelectFromModel` поверх `Random Forest` с подбором `max_features` по сетке `[5, 10, 20, 30, 40, 50, None]`.

Оптимальным значением по кросс-валидации оказалось `None` — **использование полного набора признаков**. Это содержательный результат: при слабом распределённом сигнале каждый признак вносит небольшой вклад, и полный набор работает лучше любого сокращённого. Дополнительный фактор — градиентный бустинг выполняет неявную селекцию самостоятельно, просто не выбирая шумовые признаки для разбиений, поэтому внешний отбор оказывается избыточным.

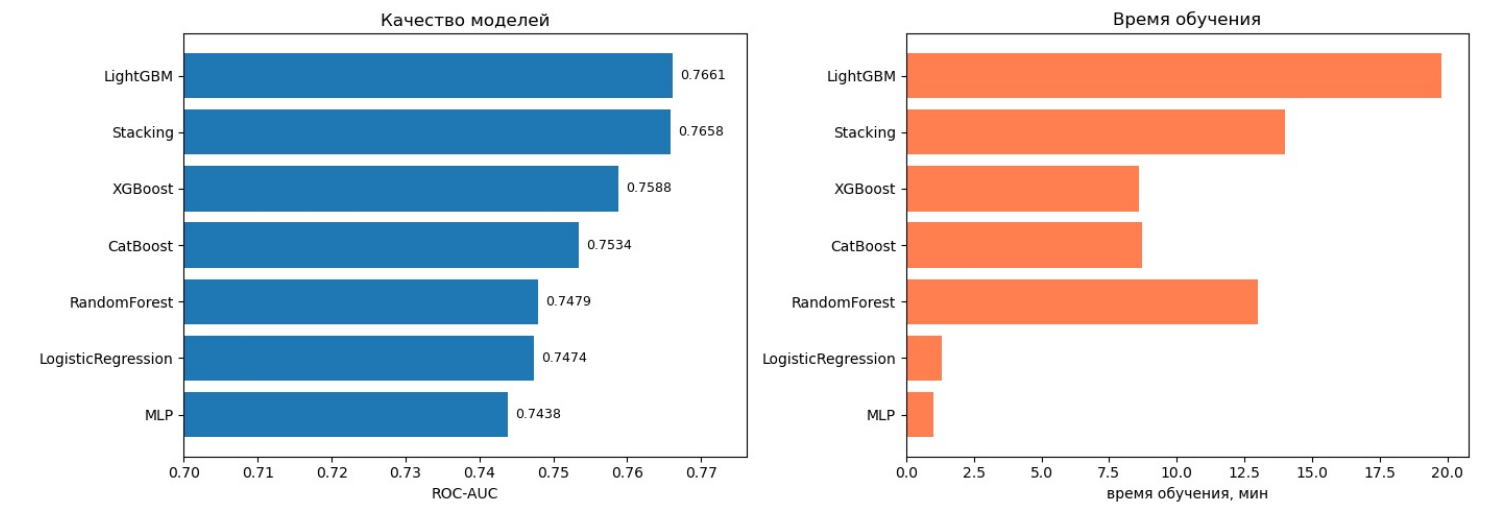
Подбор гиперпараметров: `GridSearchCV` для числа отбираемых признаков на `LightGBM` и `RandomizedSearchCV` (5 случайных комбинаций из 90 возможных) для параметров регуляризации `Random Forest`. Случайный поиск выбран из-за размера сетки: полный перебор на 307 тысячах строк потребовал бы неприемлемого времени. Лучшая конфигурация леса: `max_depth=20`, `min_samples_leaf=10`, `min_samples_split=5`.

4. Результаты

4.1 Сводная таблица

Модель	Тип	ROC-AUC	Время обучения
LightGBM	Градиентный бустинг	0.7661	19.8 мин
Стекинг (LGBM + CatBoost + XGB)	Ансамбль 2-го уровня	0.7658	14.0 мин
XGBoost	Градиентный бустинг	0.7588	8.6 мин
CatBoost	Градиентный бустинг	0.7534	8.7 мин
Random Forest	Бэггинг деревьев	0.7479	13.0 мин
Логистическая регрессия	Линейная модель	0.7474	1.3 мин
MLP (109-32-16-1)	Нейронная сеть	0.7432	0.6 мин

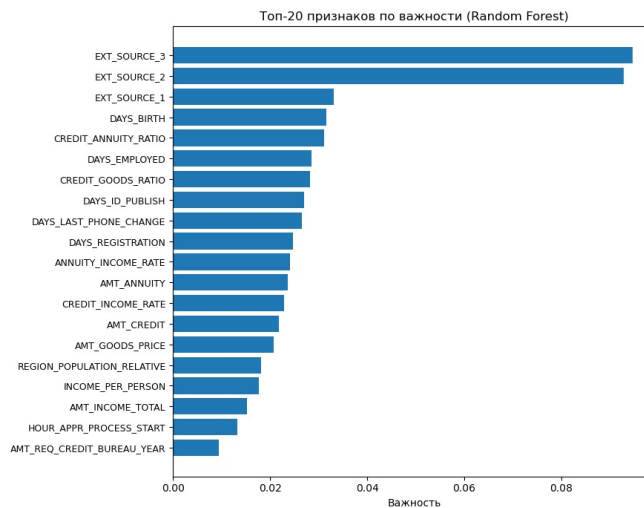
Значения *ROC-AUC* получены на 5-фолдовой стратифицированной кросс-валидации, кроме стекинга (отложенная выборка 25%) и *MLP* (единственный валидационный сплит). Стандартное отклонение по фолдам составляет 0.0045–0.0049 для всех моделей, оценивавшихся кросс-валидацией.



Сравнение моделей по качеству и времени обучения

Разброс между лучшей и худшей моделью — всего 0.022 ROC-AUC. При стандартном отклонении по фолдам около 0.005 это означает, что разница между соседними строками таблицы находится на границе статистической значимости, и содержательно интереснее не порядок моделей, а соотношение качества и вычислительной стоимости.

4.2 Важность признаков



Топ-20 признаков по важности Random Forest

`EXT_SOURCE_3` и `EXT_SOURCE_2` доминируют с большим отрывом — их важность (~ 0.093) втрое превышает важность следующего признака. Это объяснимо: внешние рейтинги уже являются агрегированными предсказаниями других скоринговых моделей, построенных на кредитной истории.

Далее следует плотная группа признаков с близкой важностью 0.02–0.03, включающая все пять сконструированных отношений и временные характеристики (`DAYS_BIRTH`, `DAYS_EMPLOYED`, `DAYS_ID_PUBLISH`, `DAYS_REGISTRATION`). Отсутствие выраженной иерархии в этой группе — ещё одно проявление распределённости сигнала.

4.3 Нейронная сеть

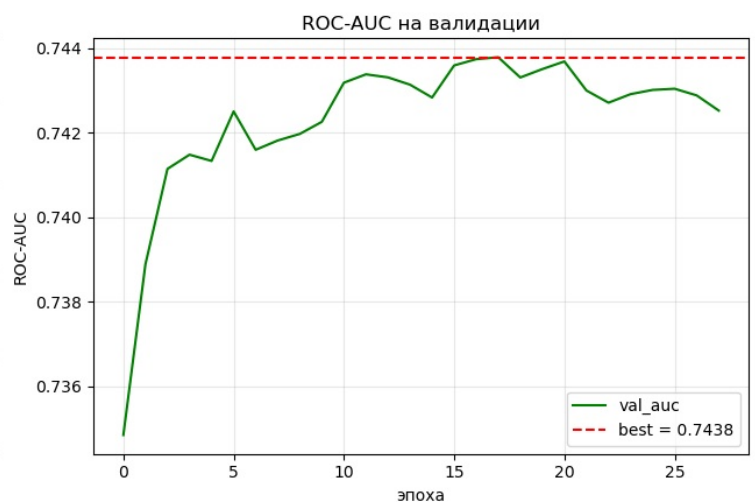
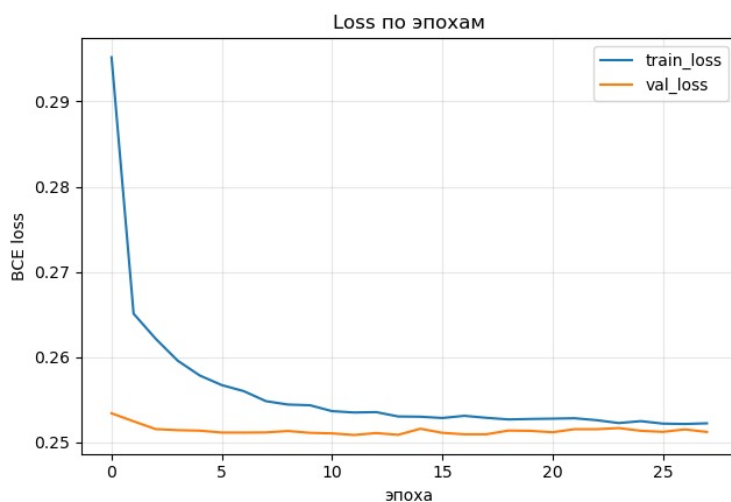
Архитектура: полносвязная сеть $109 \rightarrow 32 \rightarrow 16 \rightarrow 1$.

Обоснование выбора:

- **Два скрытых слоя.** Табличные данные не требуют глубоких сетей: основная часть зависимостей улавливается одним-двумя нелинейными преобразованиями.
- **Сужение $32 \rightarrow 16$.** Структура «воронки»: первый слой строит комбинации исходных признаков, второй сжимает их в компактное представление перед классификацией.
- **ReLU** в скрытых слоях — не страдает от затухания градиента в отличие от сигмоиды.
- **Выход без активации** — один логит, поскольку используется `BCEWithLogitsLoss`, численно более устойчивая, чем связка `Sigmoid` + `BCELoss`.

Регуляризация: `Dropout(0.3)` после каждого скрытого слоя и ранняя остановка по `val_auc` с `patience=10` и восстановлением весов лучшей эпохи.

Оптимизация: Adam, `lr=0.001`, батч 256. Обучение шло на GPU Apple Silicon (`mps`), одна эпоха занимала около 2 секунд. Ранняя остановка по `val_auc` сработала штатно и зафиксировала веса лучшей эпохи.



Кривые обучения MLP

Графики подтверждают, что **регуляризация отработала как задумано: переобучения нет**. `train_loss` идет выше `val_loss` на протяжении всего обучения — это прямое следствие Dropout, который активен на обучении и зашумляет предсказания, а при валидации отключается. Обе кривые монотонно сходятся к общему плато около 0.251 и не расходятся, то есть классического разрыва train/val, характерного для переобучения, не возникает.

Валидационный ROC-AUC быстро выходит на плато и стабилизируется, дальнейшие колебания лежат в четвертом знаке. Ранняя остановка корректно зафиксировала момент выхода на плато, избавив от лишних эпох: всё обучение уложилось в 37 секунд — быстрее любой другой модели в сравнении.

Комбинация Dropout и ранней остановки дала устойчивую модель без переобучения, а компактная архитектура обеспечила лучшее в сравнении время обучения. Естественное направление дальнейших экспериментов — расширение ёмкости сети (больше нейронов, третий слой) с соответствующей подстройкой силы регуляризации.

4.4 Стекинг

Мета-модель — логистическая регрессия поверх предсказаний трёх бустингов. Простая мета-модель выбрана намеренно: её задача — найти оптимальную взвешенную комбинацию базовых предсказаний, а не строить новые сложные зависимости, иначе она переобучится. `StackingClassifier` генерирует мета-признаки через внутреннюю 3-фолдовую кросс-валидацию, поэтому обучение мета-модели идёт на out-of-fold предсказаниях и утечки не возникает.

Результат: **0.7658**, вплотную к лучшей одиночной модели (0.7661 у LightGBM).

5. Замечания

1. Разрыв между линейной моделью и бустингом невелик, а разница в вычислительной стоимости — существенна. Логистическая регрессия даёт 0.7474 за 77 секунд, LightGBM — 0.7661 за 1188 секунд. Прирост качества составляет **+0.019 ROC-AUC при пятнадцатикратной разнице во времени обучения**. Это говорит о том, что сигнал в данных преимущественно линеен: нелинейности и взаимодействия признаков существуют, но дают умеренную добавку поверх взвешенной суммы. Для продакшена такое соотношение делает логистическую регрессию серьёзным кандидатом — она обучается за минуту, тривиально интерпретируется через коэффициенты и по регуляторным требованиям к объяснимости решений в кредитовании часто предпочтительна.

2. Random Forest показывает результат, близкий к логистической регрессии (0.7479 против 0.7474). Причина в природе алгоритма: лес усредняет независимо построенные деревья и снижает **дисперсию** модели, но не её смещение. Если отдельное дерево систематически не улавливает зависимость, усреднение сотни таких деревьев её не найдёт. Бустинг работает иначе — каждое следующее дерево обучается на ошибках предыдущих, что снижает именно **смещение**, и на задаче со слабым распределённым сигналом это оказывается решающим фактором.

3. Различия между тремя бустингами объясняются настройкой, а не архитектурой. LightGBM (0.7661) обучался с `learning_rate=0.05` и 350 деревьями, XGBoost (0.7588) и CatBoost (0.7534) — с `learning_rate=0.01` и 500 деревьями. Произведение `lr × n_estimators`, грубо характеризующее суммарный «пройденный путь» оптимизации, составляет 17.5 против 5, то есть три модели сравниваются при разном бюджете обучения. При выравнивании этого параметра разница между современными реализациями бустинга на табличных данных обычно оказывается минимальной.

Дополнительный фактор: LightGBM оценивался по `best_score` из `GridSearchCV`, то есть по максимуму из 7 конфигураций, а остальные бустинги — одним прогоном `cross_val_score`. Максимум по нескольким попыткам статистически смещён вверх, что тоже вносит вклад в его позицию в таблице.

4. Стекинг подтвердил согласованность базовых моделей. Ансамбль второго уровня даёт тем больший выигрыш, чем сильнее базовые модели ошибаются **на разных объектах**. Три градиентных бустинга, обученных на одних и тех же признаках, дают близкие предсказания — их ошибки скоррелированы, и результат ансамбля (0.7658) закономерно совпадает с лучшей одиночной моделью. Естественное продолжение — включение в ансамбль моделей принципиально иной природы: логистической регрессии и MLP, чей характер ошибок отличается от древесных моделей.

5. Отбор признаков подтвердил ценность полного набора. Подбор `max_features` по сетке показал, что оптимально использовать все признаки. При слабом распределённом сигнале каждый признак вносит небольшой вклад, и полный набор работает лучше любого сокращённого. Благодаря этому MLP получил все 109 числовых и бинарных признаков, а не урезанное подмножество.

6. Наиболее информативны внешние кредитные рейтинги. `EXT_SOURCE_1/2/3` доминируют и по корреляции, и по важности с трёхкратным отрывом. Отсюда следует практическое наблюдение, значимое для исходной бизнес-задачи: `EXT_SOURCE_1` доступен для 44% выборки, `EXT_SOURCE_3` — для 80%, то есть для части заёмщиков модель опирается на другие признаки. Для микрофинансирования, ориентированного на клиентов с тонкой кредитной историей, это указывает на перспективность отдельной модели для сегмента без внешних рейтингов — с опорой на демографию, параметры кредита и сконструированные отношения долговой нагрузки.

7. Все пять сконструированных признаков вошли в топ-20 по важности. `CREDIT_ANNUITY_RATIO` занял 5-е место — выше, чем `AMT_ANNUITY` (12-е) и `AMT_CREDIT` (14-е), из которых он построен. Это подтверждает исходную гипотезу: в кредитном скоринге относительные величины информативнее абсолютных сумм, и feature engineering, основанный на предметном понимании задачи, даёт признаки, конкурирующие с лучшими исходными переменными датасета.

6. Итоги

Построен полный пайплайн бинарной классификации для задачи кредитного скоринга: **307 511 объектов, 120 признаков, доля положительного класса 8.07%**, линейные корреляции признаков с целевой переменной не превышают 0.18 по модулю.

- Проведён EDA, обработка пропусков (67 колонок) и выбросов, кодирование категориальных признаков, feature engineering — все 5 сконструированных признаков вошли в топ-20 по важности.
- Вся предобработка оформлена через `Pipeline` и `ColumnTransformer` с отдельными стратегиями для бинарных, числовых и категориальных признаков, что исключает утечку данных на кросс-валидации.
- Обучены и сравнены **LightGBM, CatBoost, XGBoost, Random Forest, Logistic Regression, MLP и stacking-ансамбль** на стратифицированной 5-фолдовой кросс-валидации; подбор гиперпараметров выполнен через `GridSearchCV` и `RandomizedSearchCV`, отбор признаков — через `SelectFromModel`.
- Нейросеть реализована на PyTorch с Dropout и ранней остановкой, обучение на GPU Apple Silicon (`mps`).
- Лучший результат: **ROC-AUC 0.7661** (LightGBM); стекинг трёх бустингов — 0.7658. Логистическая регрессия достигает 0.7474 за 77 секунд против 1188 секунд у LightGBM, что даёт ориентир по соотношению качества и вычислительной стоимости.

Применённые навыки: pandas, numpy, matplotlib, scikit-learn (Pipeline, ColumnTransformer, кросс-валидация, подбор гиперпараметров, отбор признаков, стекинг), LightGBM, CatBoost, XGBoost, PyTorch, EDA, feature engineering, работа с несбалансированными выборками, анализ важности признаков.